Assignment 3: Calculate Pi with Monte Carlo Methods

Matthew Lochridge

February 2026

1 Pure Python

To enforce a maximum of one command line argument for the number of samples N and include a default value of N in case of no argument, the following code utilizing the sys module was added at the start of the main function of each Python method.

```
if len(sys.argv) == 1:
    n = int(1e6) # Default number of points
elif len(sys.argv) == 2:
    n = int(float(sys.argv[1])) # Command-line argument
else:
    print("Usage: python pi_numpy.py <n>")
    sys.exit(1)
```

To produce the exploratory timing table for this method (as well as the NumPy and Cython methods), the time module was used.

The time() function was called immediately before and after the sampling loop to determine the sample rate and sample time. To determine the total execution time, the time() function was called immediately before and after the calc_pi function call in the main function.

A Bash script was written to run each method iteratively, with N increasing in powers of ten up to a given maximum, and produce the table as terminal output. The exploratory timing table for this method was produced with $N_{\max} = 10^7$, for which the runtime was about 5 seconds. This table is shown as terminal output in Fig. [1].

```
Iterating pi_python.py up to 10^7 samples:
Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
2.800000        1e+01   4.196167e-05    2.621440e+05    3.814697e-06
3.160000        1e+02   7.152557e-05    1.441342e+06    6.937981e-07
3.148000        1e+03   6.163120e-04    1.639040e+06    6.101131e-07
3.130800        1e+04   4.868507e-03    2.055628e+06    4.864693e-07
3.143200        1e+05   4.938936e-02    2.024962e+06    4.938364e-07
3.143000        1e+06   5.269821e-01    1.897649e+06    5.269678e-07
3.141555        1e+07   5.331419e+00    1.875678e+06    5.331406e-07
```

Figure 1: Terminal log showing the exploratory timing table for the pure Python method with $N_{\max} = 10^7$.

2 NumPy Arrays

The exploratory timing table for the NumPy method was produced with $N_{\max} = 10^{10}$, as shown in Fig. [2]. As expected, NumPy runs faster than pure Python. The total execution time scales approximately linearly, though the sample rate appears to approach some limiting value on the order of 10^7. For $N = 10^{10}$ the program crashed because it ran out of memory when allocating the vectors of random double-precision floats; my computer has 63.2 GB of usable RAM, and a vector of 10^{10} doubles would take up 74.5 GB of memory.

```
Iterating pi_numpy.py up to 10^10 samples:
Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
2.000000        1e+01   1.587868e-04    6.502800e+04    1.537800e-05
3.080000        1e+02   1.740456e-04    5.890880e+05    1.697540e-06
3.176000        1e+03   1.559258e-04    6.584465e+06    1.518726e-07
3.140800        1e+04   6.392002e-04    1.576214e+07    6.344318e-08
3.140120        1e+05   6.690025e-03    1.496096e+07    6.684065e-08
3.139604        1e+06   4.283667e-02    2.334942e+07    4.282761e-08
3.141470        1e+07   3.803463e-01    2.629275e+07    3.803329e-08
3.141770        1e+08   3.452034e+00    2.896855e+07    3.452019e-08
3.141601        1e+09   3.329876e+01    3.003116e+07    3.329875e-08
Traceback (most recent call last):
  File "/mnt/c/Users/mdloc/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 3/pi_numpy.py", line 22, in <module>
    pi_est, sample_time = calc_pi_numpy(n)
                          ~~~~~~~~~~~~^^^
  File "/mnt/c/Users/mdloc/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 3/pi_numpy.py", line 8, in calc_pi_numpy
    h = sum(rand(n)**2 + rand(n)**2 < 1.)
            ~~~~^^^
  File "numpy/random/mtrand.pyx", line 1238, in numpy.random.mtrand.RandomState.rand
  File "numpy/random/mtrand.pyx", line 441, in numpy.random.mtrand.RandomState.random_sample
  File "numpy/random/_common.pyx", line 310, in numpy.random.__common.double_fill
numpy._core._exceptions._ArrayMemoryError: Unable to allocate 74.5 GiB for an array with shape (10000000000,) and data type float64
```

Figure 2: Terminal log showing the exploratory timing table for the NumPy method, including breakdown at $N = 10^{10}$.

3 Numba-accelerated Python

Fig. [3] shows the exploratory timing tables for the serial and parallel Numba methods. Since Python's time module is not supported within JIT compilation, total execution time was recorded in the main function, and average sample time was approximated based on that. For serial computation the initial overhead at runtime is about 0.6 seconds, and the computation time becomes significant around $N = 10^7$. For parallel computation the initial overhead is about 1.3

seconds, and the computation time becomes significant around $N = 10^9$. My computer has 16 cores, and the parallel speedup for $N = 10^9$ is about 10.5x.

```
Iterating pi_numba.py in serial mode up to 10^9 samples:
Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.200000        1e+01   6.292319e-01    1.600000e+01    6.292319e-02
3.280000        1e+02   5.846286e-01    1.710000e+02    5.846286e-03
3.136000        1e+03   6.265285e-01    1.596000e+03    6.265285e-04
3.150400        1e+04   5.905089e-01    1.693500e+04    5.905089e-05
3.146640        1e+05   6.112192e-01    1.636070e+05    6.112192e-06
3.141684        1e+06   6.039274e-01    1.655828e+06    6.039274e-07
3.142080        1e+07   9.557259e-01    1.046325e+07    9.557259e-08
3.141555        1e+08   3.467935e+00    2.883561e+07    3.467935e-08
3.141549        1e+09   2.710953e+01    3.688740e+07    2.710953e-08
```

```
Iterating pi_numba.py in parallel mode up to 10^10 samples:
Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
2.800000        1e+01   1.117791e+00    9.000000e+00    1.117791e-01
3.120000        1e+02   1.090824e+00    9.200000e+01    1.090824e-02
3.064000        1e+03   1.155428e+00    8.650000e+02    1.155428e-03
3.141200        1e+04   1.073507e+00    9.315000e+03    1.073507e-04
3.139000        1e+05   1.278610e+00    7.821000e+04    1.278610e-05
3.141868        1e+06   1.083382e+00    9.230360e+05    1.083382e-06
3.142334        1e+07   1.173783e+00    8.519460e+06    1.173783e-07
3.141533        1e+08   1.418614e+00    7.049136e+07    1.418614e-08
3.141609        1e+09   2.575710e+00    3.882425e+08    2.575710e-09
3.141610        1e+10   6.767646e+00    1.477619e+09    6.767646e-10
```

Figure 3: Terminal logs showing exploratory timing tables for the Numba method run serially (up to $N_{\max} = 10^9$) and in parallel (up to $N_{\max} = 10^{10}$).

4 Pre-compiled Cython

The exploratory timing table for the Cython method shown in Fig. [4] was produced with $N_{\max} = 10^9$, for which the runtime was about 20 seconds. There does not appear to be startup overhead as with Numba.

In this method as well as the C++ method, N and its corresponding counters in the for loops were stored as long long ints, since regular ints cannot store large N.

5 C++ Implementation

To enforce a maximum of one command line argument for the number of samples N and include a default value of N in case of no argument, the following

```
Iterating test_cython.py up to 10^9 samples:
Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.200000        1e+01   4.291534e-06    8.388608e+06    1.192093e-07
3.120000        1e+02   1.645088e-05    3.226388e+07    3.099442e-08
3.132000        1e+03   3.337860e-05    4.821039e+07    2.074242e-08
3.171200        1e+04   2.000332e-04    5.059474e+07    1.976490e-08
3.141520        1e+05   2.027512e-03    4.966612e+07    2.013445e-08
3.141664        1e+06   2.023458e-02    4.945530e+07    2.022028e-08
3.141130        1e+07   2.015064e-01    4.962745e+07    2.015014e-08
3.141698        1e+08   2.013350e+00    4.966906e+07    2.013326e-08
3.141604        1e+09   2.013791e+01    4.965765e+07    2.013788e-08
```

Figure 4: Terminal log showing the exploratory timing table for the Cython method with $N_{\max} = 10^9$.

code was added at the start of the main function of main.cpp.

```
long long int n;
if (argc == 1) {
    n = int(1e6); // Default number of points
}     else if (argc == 2)
    n = int(std::atof(argv[1]));
}
else {
    std::cerr << "Usage: " << argv[0] << " [n]" << std::endl;
    return 1;
}
```

For timing the calculations, the high_resolution_clock from the chrono header was used in the same way as the time module for the Python methods. The following lines of code extract the start time, end time, and duration to microsecond resolution.

```
auto start = std::chrono::high_resolution_clock::now(); // Start timer
...
auto end = std::chrono::high_resolution_clock::now(); // End timer
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end-start).count() * 1e-6; // Duration in seconds
```

Fig. [5] shows the exploratory timing table for the C++ method with $N_{\max} = 10^9$. Evidently the C++ method is slower than the parallel Numba method at this scale, with total execution time around 16.5 seconds for $N = 10^9$.

```
Iterating ./mc_pi up to 10^9 samples:
Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
2.800000        1e+01   1.900000e-05    2.500000e+06    4.000000e-07
3.080000        1e+02   2.000000e-05    2.000000e+07    5.000000e-08
3.044000        1e+03   3.300000e-05    5.555556e+07    1.800000e-08
3.096400        1e+04   1.610000e-04    6.849315e+07    1.460000e-08
3.137600        1e+05   1.421000e-03    7.112376e+07    1.406000e-08
3.141024        1e+06   2.271100e-02    4.406257e+07    2.269500e-08
3.142155        1e+07   1.465870e-01    6.822632e+07    1.465710e-08
3.141459        1e+08   1.479562e+00    6.758826e+07    1.479547e-08
3.141594        1e+09   1.646574e+01    6.073223e+07    1.646572e-08
```

Figure 5: Terminal log showing the exploratory timing table for the C++ method with $N_{\max} = 10^9$.

6 Comparison of Methods

A Bash script was written to run each method once for a given N and compare timing results. Fig. [6] shows exploratory timing tables as console output.

The comparison was first run with $N = 10^7$ including all methods. At this scale C++ was the fastest method, followed closely by Cython and then NumPy; the computation time for the Numba methods was outweighed by the startup overhead.

Then the comparison was run with $N = 10^9$ excluding the pure Python method. At this scale, parallel Numba was the fastest, outperforming C++ by about a factor of 7.

7 Scaling Tests and Analysis

Comprehensive scaling tests were done with the parallel Numba method. A separate Python script was written based on pi_numba.py to iterate from $N = 10^3$ to $N = 10^9$, optionally taking a command line argument for how many trials to run for each N. For each N, the computation was run 10 times, and the π estimate, total execution time, and time per sample were averaged over those 10 trials. Additionally, the absolute error in the averaged value of π, the standard deviation in the 10 trials, and the average sample rate were computed for each N. All arrays were pre-allocated to avoid additional overhead during the computation. A tab-separated .txt file containing the comprehensive benchmark table was produced at the end of the script, as shown in Fig. [7].

A Jupyter notebook was written to read in the comprehensive benchmark table and perform analysis using the NumPy and MatPlotLib modules. Fig. [8] shows the code used to perform analysis as well as the console output. Fig. [9] shows

```
Executing each method with 1e7 samples:
Pure Python: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.142002        1e+07   6.455108e+00    1.549163e+06    6.455098e-07
NumPy: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141550        1e+07   3.723779e-01    2.685522e+07    3.723671e-08
Serial Numba: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141805        1e+07   1.338563e+00    7.470696e+06    1.338563e-07
Parallel Numba: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141426        1e+07   1.104136e+00    9.056855e+06    1.104136e-07
Cython: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141130        1e+07   2.112963e-01    4.732834e+07    2.112899e-08
C++: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.142048        1e+07   1.527270e-01    6.548274e+07    1.527120e-08
```

```
Executing each method with 1e9 samples:
NumPy: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141623        1e+09   3.767286e+01    2.654432e+07    3.767284e-08
Serial Numba: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141596        1e+09   3.436952e+01    2.909555e+07    3.436952e-08
Parallel Numba: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141646        1e+09   2.141704e+00    4.669179e+08    2.141704e-09
Cython: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141604        1e+09   2.015213e+01    4.962259e+07    2.015211e-08
C++: Pi estimate, Samples, Total time (s), Sample rate (1/s), Sample time (s)
3.141699        1e+09   1.437592e+01    6.956083e+07    1.437591e-08
```

Figure 6: Terminal logs showing exploratory benchmark tables comparing performance of different methods at $N = 10^7$ and $N = 10^9$.

```
n   pi_estimate absolute_error  standard_deviation  runtime time_per_sample samples_per_sec
1000    3.1408  0.0007926535897930798   0.05625086665999028 0.16821937561035155 0.00016821937561035155  5944.618426811377
10000   3.13636 0.005232653589793301    0.01884203810631958 0.00024981498718261 72   2.4981498718261 72e-08   40029623.97404084
100000  3.141732    0.00013934641020707517  0.004758219835190426    0.00041000843048095704  4.1000843048095704e-09  243897423.96929696
1000000 3.1422096   0.0006169464102070421   0.0014113870624318228   0.0032143592834472656   3.2143592834472654e-09  311103990.5058597
10000000    3.1416786   8.594641020698 29e-05   0.0004481131017946021   0.013907790184020996    1.3907790184020994e-09  719021488.5100329
100000000   3.141641616 4.896241020668768e-05   9.914549986755623e-05   0.10492486953735351 1.0492486953735535e-09   953062895.7741973
1000000000  3.1416024116    9.758010206972756e-06   5.483637842018626e-05   0.9937860012054444  9.937860012054442e-10   1006252854.0219104
```

Figure 7: Tab-separated .txt file containing the comprehensive benchmark table for the parallel Numba method, averaging 10 estimates of π per sample size N.

a plot of absolute error and standard deviation vs. N, including the power law fit for the standard deviation.

The analysis illustrates that achieving convergence to high precision (on the order of 10^{-12}) with the fastest Monte Carlo methods examined would require utterly unreasonable amounts of memory (about 6 yottabytes, more RAM than my computer has by nearly 14 orders of magnitude) and execution time (nearly 200 million years for serial execution). The serial sample rate for C++ with $N = 10^9$ from Fig. [6] was used for estimation, since C++ was the fastest serial method for this largest N tested. Runtime can be reduced by parallelization; however, reducing the execution time to a timescale reasonable for a person to carry out, e.g. 1 year, would require about 200 million parallel processors (ideally, parallel time = serial time / number of processors, though there would be immense overhead at runtime to initialize that many parallel jobs).

```python
# fit log-log data for standard deviation
logn = np.log(n)
logsigma = np.log(pi_std)

# perform linear regression using numpy polyfit with covariance
coeffs, cov = np.polyfit(logn, logsigma, 1, cov=True)
alpha = -coeffs[0]
logA = coeffs[1]
A = np.exp(logA)

# compute uncertainties from covariance matrix
slope_err = np.sqrt(cov[0,0])
intercept_err = np.sqrt(cov[1,1])
alpha_err = slope_err
A_err = A * intercept_err  # propagate from logA

print(f"Fitted A = {A:.3f} ± {A_err:.3f}")
print(f"Fitted alpha = {alpha:.3f} ± {alpha_err:.3f}")
print(f"Relative error in alpha = {abs(alpha-0.5)/0.5:.2%}")

# create smooth curve for the fit
n_line = np.linspace(n.min(), n.max(), 100)
sigma_fit = A * n_line**(-alpha)

# extrapolate to 12 digits of accuracy (~1e-12 error)
target = 1e-12
n_req = (A/target)**(1/alpha)
mem_req_gb = n_req * 16 / 1e9  # 2 float64 values (16 bytes) per sample
print(f"Estimated samples for {target:.0e} error = {n_req:.2e}")
print(f"Memory required for {n_req:.2e} samples = {n_req*16/1e9:.2e} GB")
print(f"Memory required relative to RAM on this computer = {100*mem_req_gb/64:.2e} %")

cpp_sample_rate = 6.956083e7 # sample rate recorded with C++ method for n = 1e9
serial_time_req = n_req / cpp_sample_rate
print(f"Estimated serial time for {n_req:.2e} samples using C++ method = {serial_time_req:.2e} seconds = {serial_time_req/(365.25*24*3600):.2e} years")
print(f"Number of cores required to parallelize C++ method for runtime of 1 year = serial time / 1 year = {serial_time_req/(365.25*24*3600):.2e}")
```

```
Fitted A = 2.038 ± 0.497
Fitted alpha = 0.522 ± 0.017
Relative error in alpha = 4.41%
Estimated samples for 1e-12 error = 3.80e+23
Memory required for 3.80e+23 samples = 6.07e+15 GB
Memory required relative to RAM on this computer = 9.49e+15 %
Estimated serial time for 3.80e+23 samples using C++ method = 5.46e+15 seconds = 1.73e+08 years
Number of cores required to parallelize C++ method for runtime of 1 year = serial time / 1 year = 1.73e+08
```

Figure 8: Jupyter code and console output for analysis of the comprehensive benchmark table.

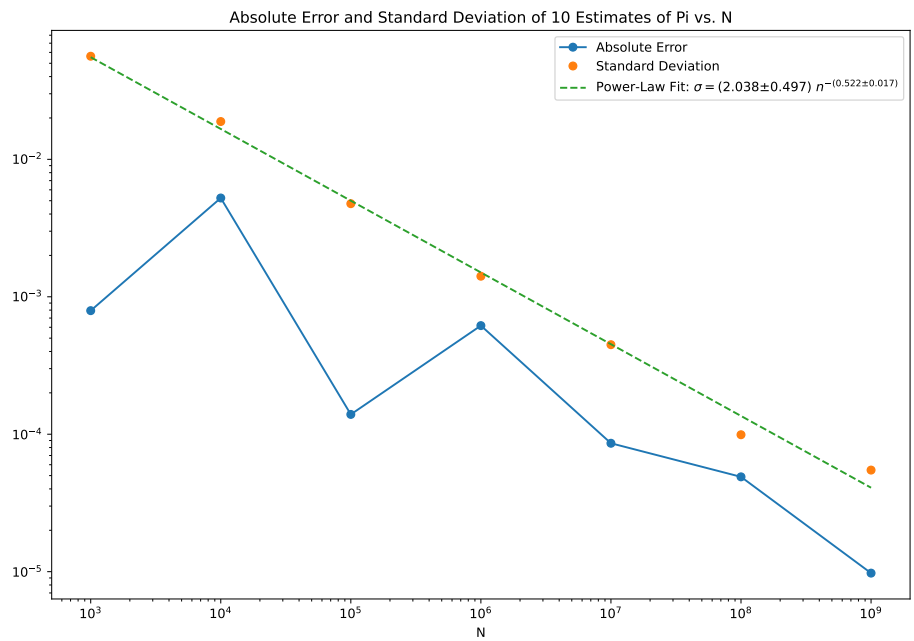

Figure 9: Plot of absolute error in average π estimate and standard deviation of estimates vs. N, produced from the comprehensive benchmark table and subsequent analysis.